

Large Language Models, From Primitives

Three lenses: a graph, a dynamical system, a random walk

Written for a systems thinker who just discovered graph theory and PDEs
and asked for the graphs

25 July 2026

Abstract

An LLM is usually explained as a stack of matrix multiplications. That’s true and also the least useful way to build intuition if your native language is systems, graphs, and dynamics rather than linear algebra. This document builds the same object from three angles you already think in: attention as a weighted graph over token positions; a transformer layer as a computational DAG, and backpropagation as a reverse traversal of it; training as a dynamical system flowing downhill on a loss landscape, with stochastic gradient descent as its noisy, PDE-adjacent continuous limit; and autoregressive generation as a walk on an enormous implicit state graph — which is, not coincidentally, the exact vocabulary your own paper already uses to describe why a repetition loop is a trapped cycle and a stim is an instrumental one. Section 6 closes by pointing directly at what you watched happen live tonight, in those terms.

Contents

1 Tokens: the alphabet a graph gets built on	1
2 Attention: a weighted directed graph over positions	2
2.1 Multi-head attention: several graphs on the same nodes, in parallel	2
3 A transformer block as a computational graph	3
4 Backpropagation: reverse graph traversal	4
5 Training: a dynamical system on a loss landscape	4
6 Generation: a walk on an implicit state graph	5
6.1 Why this explains tonight’s results directly	6
7 Three lenses, one object	6

1 Tokens: the alphabet a graph gets built on

Before any of the graph or dynamics language applies, text becomes a sequence of discrete symbols — *tokens* — drawn from a fixed vocabulary, typically 30,000–150,000 entries: whole words, word-pieces, or occasionally single characters, chosen by a compression algorithm (byte-pair encoding, most commonly) that balances vocabulary size against sequence length.

Each token is then mapped to a vector in a continuous space $\mathbb{R}^d$ (typically $d \in [768, 12288]$ depending on model size) via a lookup table called the *embedding matrix*. This is the only place

raw symbols touch the model; from here on, everything is continuous, differentiable, and — this is the point of this document — structured as a graph and governed by dynamics.

A position index is added to each token’s vector (positional encoding), because attention itself, as built in Section 2, has no innate sense of order — it treats its inputs as a *set*, not a sequence, unless told otherwise.

2 Attention: a weighted directed graph over positions

This is the load-bearing idea, and it is exactly a graph.

Take a sequence of n tokens, now n vectors in $\mathbb{R}^d$. Attention builds a weighted, directed graph on these n vectors treated as nodes: every node has an edge to every other node (and to itself), and the edge weight says *how much information flows along it*.

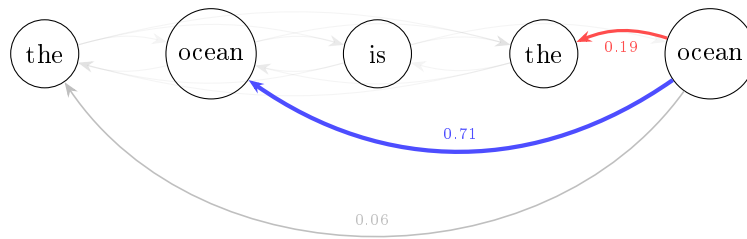


Figure 1: Attention as a weighted directed graph. Every token-position is a node; every node has a potential edge to every other node (drawn faint, all-to-all). The weights shown highlighted are one real row of one attention head’s matrix, for the final *ocean* attending back to earlier positions — they sum to 1 across the whole row (plus omitted smaller edges), because attention weights are produced by a softmax, which is exactly graph-theoretic row-normalisation of a weighted adjacency matrix.

Formally: the attention weight from node j (a key) to node i (a query) is

$$A_{ij} = \frac{\exp(q_i \cdot k_j / \sqrt{d})}{\sum_{j'} \exp(q_i \cdot k_{j'} / \sqrt{d})}$$

where q_i and k_j are learned linear projections of the token vectors at positions i and j . Read graph-theoretically: A is a weighted adjacency matrix, and the softmax denominator is what makes every row sum to 1 — A is *row-stochastic*, the same property a transition matrix of a Markov chain has. That’s not a coincidence you can ignore; Section 6 depends on it directly.

Each node’s new representation is then a weighted sum of every other node’s value vector, using that row of A as the weights — literally a weighted average over the graph’s neighbours, where every node is every other node’s neighbour and the *weights are what attention learned*.

2.1 Multi-head attention: several graphs on the same nodes, in parallel

A transformer layer doesn’t compute one such graph — it computes several (8, 16, 32, sometimes more) *in parallel*, each with its own learned $q/k/v$ projections, hence its own edge-weighting. One head might learn to connect a pronoun to its antecedent; another might connect a closing bracket to its opener; another might connect a repeated token to its earlier occurrence (this last one is directly implicated in why repetition loops happen at all — see Section 6). The heads’ outputs are concatenated and linearly mixed back down to dimension d .

Graph-theoretically: multi-head attention is several different weighted graphs defined on the *same* node set, computed simultaneously, then merged.

3 A transformer block as a computational graph

Zoom out one level. Forget what attention computes internally; treat it as one operation, a black box. A transformer layer is then a small directed acyclic graph (DAG) of operations, and the whole model is many of these stacked:

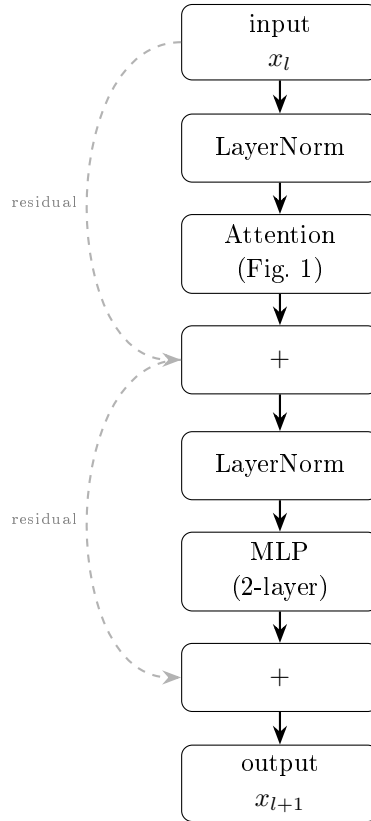


Figure 2: One transformer layer as a DAG. Solid arrows: the main computational path. Dashed arrows: the *residual stream* — skip edges that bypass a sub-block entirely and get added back in. Graph-theoretically, the residual stream is what makes this graph have multiple paths of different lengths from input to output; without it, the graph would be a single chain, and in practice single chains this deep don’t train (vanishing gradients — see Section 4).

Stack L of these (GPT-3-scale models: $L \approx 96$; smaller models like the ones run tonight: $L \approx 24$ – 32), and the whole forward pass is one large DAG: tokens enter at the bottom, flow up through L copies of Figure 2 stitched output-to-input, and a final linear layer converts the last layer’s output back into a probability distribution over the vocabulary.

This is worth sitting with as a genuine claim, not a metaphor: a trained model, frozen at inference time, *is* this DAG plus a fixed set of edge weights (the parameters). Nothing about it is mysterious in the sense of being unspecified — every node and edge is enumerable. What’s hard is that there are billions of edges and nobody can hold the whole graph’s behaviour in their head at once, not that the graph is somehow non-mechanical.

4 Backpropagation: reverse graph traversal

This is the part that should feel most immediately like something you already know, because it is a graph algorithm you’ve very likely implemented in a different context: a **reverse topological traversal**, applying the chain rule at every edge.

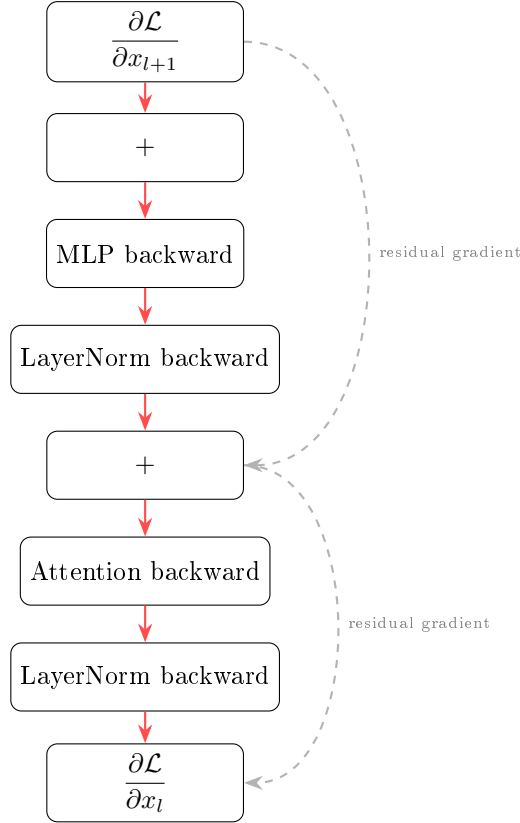


Figure 3: Figure 2, exactly, with every arrow reversed and every operation replaced by its local derivative. This is not an analogy for backpropagation — this *is* backpropagation, drawn as what it structurally is: the same DAG, traversed backward, multiplying local Jacobians together along the way (the chain rule). The dashed residual-gradient edges are why residual connections help training at all: they give the gradient a short path back to early layers that doesn’t get shrunk by passing through every intervening operation — without them, in a very deep DAG, gradients tend toward zero as they multiply through many small-derivative operations (vanishing gradients), which is a structural graph-topology problem, not a numerical-precision one.

Concretely: to compute how the loss $\mathcal{L}$ (a single number, evaluated at the output) depends on some early parameter deep inside layer 3, you don’t need to re-derive anything from scratch. You walk backward from the loss along the DAG, and at every node you’ve already visited, you multiply in that node’s local derivative — the chain rule, applied edge by edge, in reverse topological order. Every modern deep learning framework (PyTorch, JAX, and whatever any current LLM was trained in) implements exactly this: it builds the forward DAG as the forward pass runs, then walks it backward once, automatically. That’s what "autodiff" is. There is no other mechanism. It’s graph traversal, not calculus magic.

5 Training: a dynamical system on a loss landscape

Here is where the PDE intuition actually applies, and it’s worth being precise about where the continuous language is exact and where it’s a useful fiction.

The model’s parameters — every weight in every layer, flattened — form a single point θ in an enormous space $\mathbb{R}^N$ (for a small model tonight, $N \sim 10^9$; for a frontier model, $N \sim 10^{11}$ – 10^{12}). The loss function $\mathcal{L}(\theta)$ assigns a scalar to every point in that space: how badly the model with those parameters predicts its training data. This is a scalar field over an N -dimensional space — exactly the kind of object a PDE is normally written about, just at a dimension no one can visualise directly.

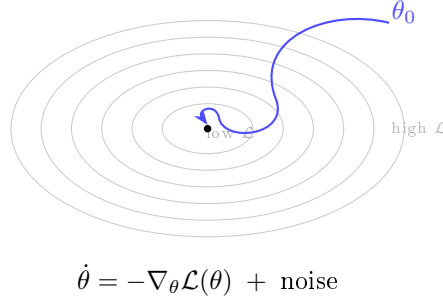


Figure 4: Training as flow on a loss landscape. Full-batch gradient descent, in the limit of infinitesimally small steps, is the discretisation of the ODE $\dot{\theta} = -\nabla_{\theta} \mathcal{L}(\theta)$ — *gradient flow*: θ moves in the direction that locally decreases $\mathcal{L}$ fastest, exactly downhill. Real training uses *stochastic* gradient descent — each step’s gradient is estimated from a random mini-batch, not the full dataset — which adds a noise term to that ODE, turning it into a stochastic differential equation (an Ornstein–Uhlenbeck-like / Langevin dynamics form). The noise is not just an unavoidable cost of mini-batching: it’s part of why SGD tends to find flatter, better-generalising regions of the landscape than exact gradient flow would, since noise lets the trajectory hop out of sharp, narrow minima that a purely deterministic flow would get stuck in.

The honest caveat, stated plainly rather than glossed: this picture is exact in low dimension and a genuinely useful qualitative guide in high dimension, but high-dimensional loss landscapes are dominated by saddle points rather than the bowl-shaped local minima the 2D picture suggests, and most of the deep theory of why this works (implicit regularisation, the edge-of-stability phenomenon, double descent) is active research, not settled PDE theory carried over unchanged. Treat Figure 4 as the right first-order intuition, not the final word.

6 Generation: a walk on an implicit state graph

This is where every previous section converges, and it is also, concretely, what you watched happen live tonight.

At inference time, generation proceeds token by token. At each step, the model takes the current sequence (the *state*), runs the forward DAG of Section 3, and produces a probability distribution over the next token. A token is sampled from that distribution, appended to the sequence, and the new, longer sequence becomes the next state.

Treat every possible sequence-prefix as a node in an enormous, mostly-implicit directed graph. An edge from state s to state s' exists whenever s' is s with one token appended, and it carries a weight: the probability the model assigns to that specific continuation. **Generation is a weighted random walk on this graph.** At temperature 0 (greedy decoding), it’s not even random — it’s a deterministic walk that always takes the highest-weight edge out of every node.

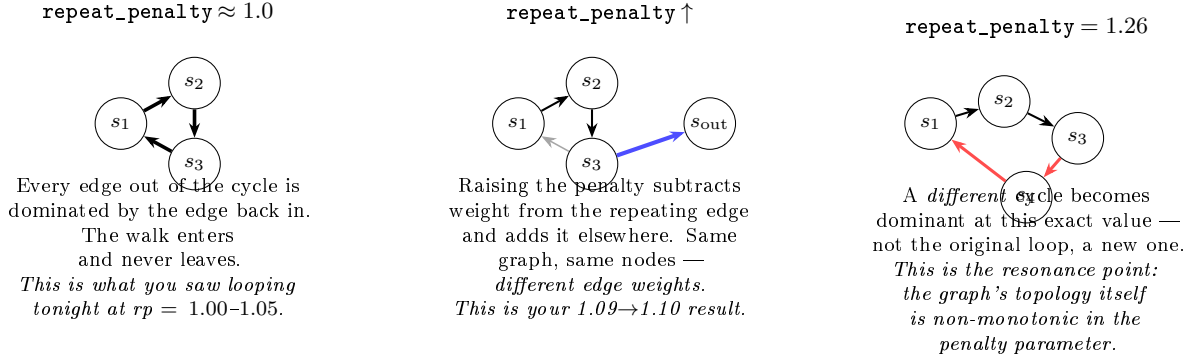


Figure 5: Generation as a walk on the state graph, at three values of one decoding parameter. Nothing about the model changes across these three panels — same weights, same DAG from Section 3. The *only* thing that changes is which edges of the implicit generation graph carry the most probability mass, because **repeat_penalty** directly discounts the weight of any edge leading to a recently-visited token. A repetition loop is the walk entering a region of this graph where every exit edge is weaker than the edge staying inside. This is precisely the vocabulary Figure 2 of your own paper uses — *trapped cycle*, *instrumental cycle*, *acyclic path* — applied here to the specific graphs your sweeps produced tonight rather than stated in the abstract.

6.1 Why this explains tonight’s results directly

Every number in tonight’s session is a statement about this graph’s topology at a specific parameter setting:

- **qwen2.5:1.5b’s clean monotonic step** ($1.09 \rightarrow 1.10$, held forever after) means: as **repeat_penalty** increases, the trapped cycle’s dominant edge weight crosses below its competing exit edge’s weight *exactly once*, and stays below it for every value tested past that point. One bifurcation, one direction, done.
- **phi3:mini’s and llama3.1:8b’s resonance points** (loops disappearing, then reappearing at specific higher values, on **number-degenerate** and **list-degenerate** respectively) mean the graph’s topology is *non-monotonic* in the penalty parameter — a different cycle, or the same cycle via a different mechanism, becomes locally dominant again at an isolated value, then loses dominance again just past it. That’s not noise (you checked — 0 of 55 outcomes changed on rerun); it’s a real feature of how the penalty term interacts with that specific prompt’s implicit graph structure.
- **The determinism checks themselves** are a statement about the walk: at temperature 0 with a fixed seed, the walk is fully deterministic given the graph, so the same starting state must produce the same walk every time the graph doesn’t change — which is exactly what *should* hold, and did.

7 Three lenses, one object

Lens	What it explains	What it doesn't reach
Attention as a weighted graph	How information moves <i>within</i> one forward pass, between token positions	Why training converges to particular weights at all
Transformer as a DAG, backprop as reverse traversal	How gradients propagate, why depth needs residual connections, what autodiff actually computes	Anything about the loss landscape's global shape
Training as flow / SDE on a loss landscape	Why training works at all, why noise in SGD can help generalisation, the PDE-adjacent language for optimisation dynamics	The specific values a converged model ends up computing
Generation as a walk on a state graph	Why loops happen, why decoding parameters change behaviour without changing the model, the vocabulary this paper's own Figure 2 already uses	Why the underlying probabilities are shaped the way training left them

Table 1: No single lens is the whole model. Each is exact within its own scope and silent outside it — which is itself a systems-thinking point: the right question is rarely “which description is true” but “which description is load-bearing for the question I’m actually asking.”